KNIME Workflows in Scientific Studies: Publication Practices and Reproducibility Risks

Vitalii Kaplan[0009-0009-8181-2863]

2333275007@alu.istinye.edu.tr

Abstract. In this paper, we study how visual workflows are reported in scientific publications and how incomplete workflow preservation can make older studies harder to reproduce as workflow platforms evolve. We use KNIME as a case study, combining OpenAlex bibliometrics, a 100-record top-cited article audit registry, workflow retrieval and current-KNIME opening/execution checks, and repository-level mining of KNIME node metadata. OpenAlex shows that KNIME is visible across a broad scientific literature, while the article assessment shows that influential studies often report KNIME through figures, descriptions, or tool mentions rather than downloadable workflow artifacts. In the 100-record article audit registry, twenty-two records report downloadable or linked KNIME workflow files. Recorded retrieval attempts obtained workflow artifacts or workflow-directory evidence for eighteen article records. In the recorded manual execution checks, five article records had at least one workflow execute successfully. Repository mining gives the source-evolution context: deprecated ordinary nodes increased from 14.83% of registered ordinary nodes in the 2018 source baseline to 33.33% in the June 28, 2026 local source snapshot. Together, these results show that weak workflow preservation and evolving node metadata can make long-term reproduction of KNIME-based studies harder over time.

Keywords: Workflow reproducibility · KNIME · Deprecated nodes · Software evolution

1 Introduction

KNIME is a widely used open-source platform for constructing scientific data-analysis workflows, with more than a decade of use across published studies. Its visual workflows represent analyses as connected nodes with stored settings, data ports, and extension dependencies, which can make computational work easier to inspect, reuse, and repeat. This representation provides more operational detail than a narrative method description, but it does not remove the need to preserve the executable workflow artifact and its context. Reproduction requires access to the workflow file, KNIME version, extensions, data, and execution environment. Figures and textual summaries do not provide sufficient substitutes for these materials.

This issue matters for visual workflow platforms in general, because their reproducibility value depends on preserving executable workflow artifacts and the software context needed to interpret them. We study KNIME because it is an open-source visual workflow platform with a long public development history and substantial visibility in scientific literature. Our OpenAlex query returned 963 article-type records whose titles or abstracts match `KNIME`. The matching records span platform papers, extension papers, tool-comparison papers, and domain studies that use KNIME as an analysis interface. KNIME is therefore a suitable case for studying whether visual workflows are preserved as executable artifacts or only reported through descriptions and figures.

Our top-cited article collection covers the 100 most-cited KNIME-matching records, with not-assessed placeholders retained where local full text is unavailable. The structured audit reported in Table 3 shows that publication practice is the weak link. Twenty-two records report downloadable or linked KNIME workflow files, and workflow artifacts or workflow-directory evidence were obtained for eighteen article records in the recorded retrieval attempts. Other links were stale, blocked, incomplete, or did not lead to a confirmed KNIME workflow artifact. In the recorded manual execution checks, five article records had at least one workflow execute successfully. For other influential studies, the strongest workflow evidence is usually weaker than an executable artifact. Several records present screenshots or figures of connected KNIME nodes, or describe workflow steps in text, without preserving all executable workflow context.

These reporting practices are risky because KNIME continues to evolve. A workflow that was executable when a paper was written may later depend on nodes that are deprecated, hidden, migrated, removed, or supplied by third-party extensions that are no longer easy to install. This paper therefore connects publication evidence with platform-evolution evidence. We combine OpenAlex bibliometrics, top-cited article retrieval and assessment, workflow-artifact retrieval and current-version execution attempts, and repository-level mining of KNIME node metadata. The goal is to show why publication practice and source-code evolution must be considered together when evaluating long-term reproducibility of KNIME-based studies.

The rest of the paper is organized as follows. Section 2 reviews related work on computational reproducibility, open-source workflow systems, environment preservation, and platform evolution. Section 3 presents the methods before the data because the study depends on how the bibliometric records, article assessments, and repository snapshots were collected. Section 4 describes the resulting datasets. Section 5 reports the bibliometric, article-audit, workflow-retrieval, manual compatibility-test, and repository-mining results. Section 6 discusses the implications, limitations, and future extensions of the study.

2 Literature Review

Reproducibility is a central requirement for computational research because published results increasingly depend on software, data transformations, and execu-

tion environments. Provenance research makes this requirement explicit: visual analytics systems need records of data, analytical processes, and human interpretation to make later inspection and reuse possible [24]. Studies of executable research artifacts show the same problem in practice. Samuel and Mietchen found that biomedical Jupyter notebooks often fail during dependency installation or re-execution even when the notebooks themselves are available [20].

Open-source scientific software helps reproducibility by making algorithms, interfaces, and execution infrastructure inspectable. Weka, for example, is presented as an extensible open-source data-mining environment with graphical interfaces, reusable configurations, and plugin mechanisms [5]. KNIME is similarly used as an open-source visual workflow platform, and recent GIScience work explicitly proposes KNIME-based workflows as an approach for advancing replicable and reproducible research [3]. However, openness of the platform alone is insufficient. A workflow-based study also requires the executable workflow file, input data, platform version, installed extensions, and other software dependencies. Container research makes the same point at the environment level: scientific computations are easier to reproduce when complete software environments can be preserved and moved between systems [14]. More broadly, the testing literature shows that executable artifacts can remain sensitive to operating systems, dependency versions, timing, external services, and other environmental factors [22].

Even when an executable workflow is published, long-term reproduction can be affected by the evolution of the workflow platform itself. KNIME's node extension schema states that a deprecated node is no longer shown in the node repository, but is still loaded in existing workflows [12]. This design supports backward compatibility, while also marking a change in the ordinary authoring surface. KNIME workflows also persist node factory class names, and the platform provides extension points for mapping old factory classes to newer implementations and for registering workflow-node migration rules [11, 13]. These mechanisms show that continued workflow loading may depend on compatibility metadata, extension availability, and migration support. We therefore treat KNIME node deprecation as a measurable risk marker for long-term workflow reproducibility.

3 Methods

All scripts and data used for this study, except the other articles assessed as study objects, are available in the public replication repository [8]. Relative paths reported in this paper should be interpreted as paths within that GitHub repository.

3.1 Bibliometric Evidence

We collected bibliometric evidence from the OpenAlex Works API [18] on June 29, 2026. The query used OpenAlex title-and-abstract search for the word `KNIME`

and restricted the result set to OpenAlex records whose type is `article`. The same query can be reproduced in the OpenAlex web interface by selecting "Search title and abstract", entering `KNIME`, and filtering the result type to `article`. The corresponding web-interface request can be represented as:

```
https://openalex.org/works
page=1
sort=relevance_score:desc
search.title_and_abstract=KNIME
filter=type:article
```

The first API request, which returns the first 200 records, is reproducible as:

```
https://api.openalex.org/works
?search.title_and_abstract=KNIME
&filter=type:article
&per-page=200
&cursor=*
```

OpenAlex uses cursor pagination for large result sets. After the first request, the collector reads `meta.next_cursor` from each response and submits the same request with `cursor=<next-cursor>` until no further results are returned. The value `per-page=200` is the maximum page size used here.

The collection script is `scripts/collect_openalex_knime_works.py`. It stores raw and processed collection artifacts as follows:

- raw OpenAlex response pages: `data/original/openalex/pages/`
- complete record stream: `data/original/openalex/works.jsonl`
- compact CSV view: `data/original/openalex/works.csv`
- endpoint, query parameters, collection time, result count, and page manifest: `data/original/openalex/metadata.json`

The current run collected 963 records across 5 OpenAlex response pages. Derived bibliometric tables are generated from the JSONL file by `scripts/build_openalex_knime_bibliometrics.py`.

3.2 Top-Cited Article Assessment

For the article-level audit we developed an open-source application composed of scripts, JSON configuration files, and prompt configuration files. The process is specified in `scripts/audit_script_chain.json`, which acts as the workflow index for this part of the study. Each step consumes the recorded outputs of earlier steps and writes an explicit intermediate or final artifact. The current chain contains fifteen scripted steps and one manual checkpoint. Thirteen scripted steps are rule-based. Two scripted steps use an LLM. We use the LLM only where rule-based extraction is insufficient. The first LLM step classifies workflow relevance from downloaded reference-page content. The second LLM step fills supplementary article-level audit flags from the processed text of the articles. The LLM

calls use temperature 0 to reduce variation. Repeated execution may still depend on the model and service version used.

The pipeline is mostly automatic, but it is deliberately semi-automatic at two boundary points: obtaining article PDFs at the beginning of the process and obtaining workflow artifacts near the end, before formal result preparation. Both tasks may involve institutional access, protected publisher pages, supplement portals, captchas, and changing web interfaces. The process can therefore be improved by human assistance at the beginning and end, while the middle steps are executed automatically from the recorded intermediate files.

The implementation was run with Python 3.14.5 and uses a separate `.venv-playwright` virtual environment for browser-dependent steps. Article PDF conversion uses GROBID `0.9.0-crf` through the Docker image `grobid/grobid:0.9.0-crf` [4]. Browser-assisted fetching uses Playwright for Python 1.61.0 with Chromium/Chrome [15]. The two LLM-supported steps call the OpenAI Chat Completions API directly, using the configured model `gpt-4.1-mini` and temperature 0 [16]. OpenAlex is the bibliographic source for article discovery and ranking [18]. The exact commands, script order, prompt files, and intermediate outputs are recorded in `scripts/audit_script_chain.json`.

For reproducibility of the LLM-supported steps, API configuration is separated from prompts and data. The scripts read the API key from `.env` or the process environment as `OPENAI_API_KEY`; this local credentials file is not part of the research data. The model can be overridden with `OPENAI_MODEL`, but the recorded run used the default `gpt-4.1-mini`. Both LLM scripts use the Chat Completions endpoint `https://api.openai.com/v1/chat/completions`, temperature `0.0`, and at most three retries for transient server-side failures. The prompts are stored as JSON configuration files: `data/processed/audit/article_reference_classification_prompt.json` for reference-page classification and `data/processed/audit/article_supplementary_flag_prompt.json` for article-level supplementary flags. The derived JSON outputs record the script name, input file, prompt configuration, model, temperature, selection scope, summary counts, and per-article classifications. This makes the exact prompt and local evidence auditable, while recognizing that future reruns may differ if the hosted model implementation changes.

Step 1 collects KNIME-matching article records from the OpenAlex works API with `scripts/collect_openalex_knime_works.py`. The query searches for KNIME in article titles and abstracts and stores the original API response pages, a complete JSONL stream, a compact CSV view, and query metadata. This step preserves the bibliographic source records before any article-level audit decisions are made.

Step 2 builds the bibliometric summaries and the citation-ranked seed table with `scripts/build_openalex_knime_bibliometrics.py`. The script reads the OpenAlex JSONL export and derives processed CSV and JSON files used to select the top-cited audit candidates. The citation-ranked seed table is then used as the article list for downstream download, parsing, URL collection, and audit report construction.

Step 3 attempts to download article PDFs by citation-rank range with `scripts/download_openalex_rank_range_articles.py`. This step is semi-automatic because OpenAlex links and open-access URLs do not always expose the full paper, and some papers require lawful institutional access. The script records download attempts and stores retrieved PDFs under `data/original/articles/`; failed or protected records can be completed by manual download without changing the later audit logic.

Step 4 converts the local PDFs into semantic article HTML with `scripts/extract_article_grobid_html.py`. The current parser uses a local GROBID service to produce TEI XML and then renders an HTML reading copy with article metadata, abstract, body sections, paragraphs, and bibliography structure. This replaced earlier plain-text extraction because the downstream steps need stable article identity, paragraphs, and reference context.

Step 5 collects article metadata and article-related URLs with `scripts/collect_article_urls.py`. The script joins the OpenAlex seed table, the canonical PDF registry, the GROBID TEI files, and the generated HTML files. At this stage it does not classify URLs or infer workflow availability; it only records metadata and normalized URLs that can be traced to the article content.

Step 6 fetches the referenced URL pages with `scripts/fetch_article_url_pages.py`. For each unique URL, the script records HTTP status, redirect chain, final URL, response headers, and a stored copy of the response body where available. This creates local evidence for later classification and avoids asking the language model to rely on live web state.

Step 7 attaches the URL-fetch metadata back to the article records with `scripts/attach_url_fetch_metadata.py`. This rule-based step augments each URL entry with fetch status, HTTP code, final URL, redirect count, and the path to any stored page. It is metadata attachment only; it does not interpret whether a reference is a workflow, software page, dataset, article landing page, or documentation.

Step 8 performs a second-stage fetch for reference pages that ordinary HTTP requests cannot access, using `scripts/browser_fetch_403_urls.py`. This step uses a browser when useful, stores rendered pages, and records challenge or blocked states explicitly. The process does not solve captchas or bypass access controls; protected pages are treated as evidence of access friction rather than forced downloads.

Step 9 classifies reference pages for workflow obtainability with `scripts/classify_article_references_with_llm.py`. The language model is given URL metadata and locally cached reference-page excerpts, not the whole article as its primary evidence. The purpose is to decide whether a reference can help a reader obtain a KNIME workflow artifact, whether it only points to software, documentation, data, or a general page, or whether manual checking is needed.

Step 10 classifies supplementary article-level flags with `scripts/classify_article_supplementary_flags_with_llm.py`. This LLM step uses the article HTML, metadata, and reference classifications to fill bounded boolean fields comparable to a manual audit checklist: KNIME use, KNIME version reporting,

workflow screenshots or figures, textual workflow or node descriptions, input-data availability, code or scripts, and extension or plugin information. These flags provide article-level context around the reference-page workflow evidence.

Step 11 builds a compact article audit report with `scripts/build_article_audit_report.py`. This rule-based join combines article metadata, the supplementary flag fields, and the workflow-relevant reference classifications into `data/processed/audit/article_audit_report.json`. The report is the central article-level audit artifact used by later workflow-download and table generation steps.

Step 12 attempts automatic workflow artifact downloads from the workflow references in the audit report with `scripts/download_workflow_artifacts.py`. The script uses rule-based HTTP retrieval and, where appropriate, browser-assisted discovery to look for KNIME workflow archives, `workflow.knime` files, or archives containing workflow files. It writes attempt status, local directories, downloaded files, and discovered workflow files to the workflow-reference inventory.

Step 13 is the manual workflow-download checkpoint. The auditor inspects `data/processed/audit/knime_downloadable_workflow_references.json` and tries to download missing workflow artifacts where access is allowed. The inventory JSON is not edited manually at this stage; the manual action is to place obtained files in the article-specific directory under `data/original/workflows/`. This preserves a clear boundary between human retrieval and scripted evidence updating.

Step 14 updates the article report from retrieved workflow evidence with `scripts/update_article_report_from_workflow_inventory.py`. The script scans the workflow inventory and the actual files under `data/original/workflows/` and updates the article audit report when a downloadable KNIME workflow has been obtained. This makes the report reflect the local evidence rather than only the earlier URL classification.

Step 15 generates the article-audit summary table source with `scripts/build_article_audit_tables.py`. The script reads the compact article audit report and writes the CSV files used for the publication-level summary table. The table data are therefore derived from the final structured report rather than maintained by hand.

Step 16 generates the KNIME-use workflow-reporting table source with `scripts/build_knime_use_workflow_reporting_table.py`. This step combines the article-level audit report with the workflow-reference inventory, so that publication-level workflow-reporting signals and actual obtained workflow evidence are kept separate but can be summarized consistently.

The resulting process is reproducible at the level of inputs, scripts, configuration files, cached pages, LLM prompts, and derived JSON reports, while still acknowledging that publication and workflow retrieval are not fully automatable. Manual actions are restricted to acquiring protected article PDFs and workflow artifacts where access is lawful; the subsequent interpretation, report updating, and table construction are scripted from recorded evidence.

3.3 Repository-Level Analysis

We study KNIME node deprecation through repository mining of the public `knime-oss` GitHub organization. KNIME Analytics Platform is a workflow-based data-analysis system in which analyses are assembled from connected processing nodes [1, 10]. KNIME extensions are distributed through update sites and extend the platform with additional functionality [9]. The KNIME source repositories were cloned locally, and each source snapshot was reconstructed by checking out every non-hidden top-level Git repository to the latest commit at or before a selected target date. This checkout step is implemented by `scripts/checkout_knime_oss_by_date.sh`. The script records a manifest for each snapshot, including repository name, checkout status, target date, selected commit, selected commit date, and previous head.

The current repository-mining corpus contains 91 immediate Git repositories in the local clone. The date-based mining process includes 90 non-hidden immediate repositories. The hidden `.github` repository is excluded because it contains organization metadata rather than KNIME node implementation code. The exact repository list is stored in `data/processed/knime_snapshots/knime_oss_repositories.csv`.

For each date-based source snapshot, we parse KNIME metadata files rather than grepping source text. The main deprecation marker is `deprecated="true"` on ordinary node registrations in `plugin.xml` under the Eclipse extension point

`org.knime.workbench.repository.nodes`

The KNIME schema states that deprecated nodes are no longer shown in the node repository but are still loaded in existing workflows [12]. Dynamic node sets use a separate extension point:

`org.knime.workbench.repository.nodesets`

We count these registrations separately. Node-description XML files ending in `NodeFactory.xml` and rooted at `knimeNode` are parsed as a secondary deprecation source. Hidden nodes, marked with `hidden="true"`, are counted separately from deprecated nodes.

The extraction also records compatibility-related metadata from two extension points:

`org.knime.core.NodeFactoryClassMapper`

`org.knime.workflow.migration.NodeMigrationRule`

Together, these extension points document mechanisms for mapping persisted node-factory class names to newer implementations and for registering workflow-node migration rules [11, 13]. These records are not counted as deprecation markers, but they are relevant for later analysis of whether deprecated or replaced nodes have explicit migration support.

The parser is implemented in `scripts/collect_knime_node_snapshot.py`. It uses Python's standard XML parser and skips `.git`, `target`, `bin`, and `.metadata`

directories. Boolean XML attributes are treated as true only when their value is case-insensitively equal to `true`.

For each snapshot, the extractor writes per-record tables for node registrations, node descriptions, factory-class mappers, migration rules, and a local summary. The per-snapshot outputs are stored under `data/original/knime_snapshots/<snapshot-date>/` and include:

- `logs/checkout_<snapshot-date>.csv`
- `plugin_nodes.csv`
- `node_descriptions.csv`
- `factory_class_mappers.csv`
- `migration_rules.csv`
- `summary.csv`

The cross-snapshot table is stored as:

```
data/processed/knime_snapshots/
knime_node_snapshot_summary.csv
```

It is generated from these per-snapshot outputs by `scripts/build_knime_node_snapshot_summary.py`.

The summary builder aggregates ordinary node registrations, dynamic nodesets, unique factory classes, deprecated nodes, hidden nodes, node-description files, deprecated node descriptions, factory-class mappers, and migration rules. It also counts processed and skipped repositories from each checkout manifest. Transition columns compare adjacent chronological snapshots. Node identity is based on `factory_class` where available. Otherwise, the fallback key is `plugin_xml:element:category_path`. These transition counts are therefore metadata-level approximations and are strongest for nodes with stable factory classes.

All project Python scripts are intended to be run with Python 3.11 or newer. The current scripts use only the Python standard library. No third-party Python packages are required for the OpenAlex and KNIME repository-mining pipelines.

4 Data

4.1 Bibliometric Evidence

The OpenAlex bibliometric data are organized into original API responses and processed analysis tables. The original data are stored under `data/original/openalex/`. They include raw response pages, a complete JSONL export, a compact CSV export, and query metadata. These files preserve the article-level records used to measure the size, time trend, venues, fields, and accessibility of the KNIME-matching literature.

Derived bibliometric tables are stored under `data/processed/openalex/`. Their roles in the analysis are:

- `openalex/openalex_knime_articles.csv`: compact article-level metadata used as the main processed bibliography table.
- `openalex/openalex_knime_articles_by_year.csv`: annual publication counts used to describe the time trend of KNIME-matching literature.
- `openalex/openalex_knime_top_venues.csv`: source-level aggregation used to identify journals, proceedings, repositories, and platforms where the matching records appear.
- `openalex/openalex_knime_top_domains.csv`: OpenAlex domain aggregation used to describe the broad disciplinary distribution.
- `openalex/openalex_knime_top_fields.csv`: OpenAlex field aggregation used to describe the finer disciplinary distribution.
- `openalex/openalex_knime_most_cited.csv`: citation-ranked subset used to select influential records for later manual assessment.
- `openalex/openalex_knime_likely_workflow_platform.csv`: heuristic subset used to prioritize papers likely to use KNIME as a workflow platform.

The processed OpenAlex directory also contains additional summary files:

- `openalex/openalex_knime_open_access_availability.json`
- `openalex/openalex_knime_role_classification_summary.json`

These files summarize artifact-access signals and role-classification counts.

4.2 Top-Cited Article Assessment

The top-cited article dataset records the structured assessment of the 100 most-cited OpenAlex records. The citation-ranked source list is `data/processed/openalex/openalex_knime_most_cited.csv`, and the local article registry is stored under `data/original/articles/`. Local PDFs, where available, are preserved in that directory, while download-attempt logs are stored under `data/processed/audit/logs/`.

Article text and references are represented by GROBID outputs. Semantic HTML files are stored under `data/processed/articles/`, and TEI XML files are stored under `data/processed/articles/grobid_tei/`. These outputs provide article text, section structure, bibliography context, and machine-readable links for the later URL and LLM steps.

The URL extraction step writes `data/processed/audit/article_url_collection.json`. For each article, this file stores rank, canonical identifiers, PDF path, TEI path, HTML path, OpenAlex metadata, and article-related URLs extracted from the GROBID outputs. The fetched reference-page cache is stored under `data/processed/audit/pages/`; browser-rendered fetches for selected blocked pages are stored under `data/processed/audit/browser_pages/`. These caches include HTTP status, redirects, final URLs, headers, stored body paths, and fetch errors or challenge pages where applicable.

The LLM-derived article-audit data are split into two files. `data/processed/audit/article_reference_llm_classifications.json` classifies cached article references by workflow obtainability and records the page evidence used for

those classifications. `data/processed/audit/article_supplementary_llm_flags.json` records article-level flags for KNIME use, version reporting, workflow figures, workflow text descriptions, data availability, code availability, and extension information.

The compact report used for the article-level tables is `data/processed/audit/article_audit_report.json`. It merges article metadata, supplementary flags, and workflow-relevant reference classifications into one record per article. Workflow-download attempts and local workflow paths are recorded in `data/processed/audit/knime_downloadable_workflow_references.json`; manual KNIME-opening screenshots are stored under each workflow directory's `opened/` subdirectory. Table sources are generated under `article/tables/` from these JSON files rather than edited manually.

4.3 Repository-Level Analysis

The KNIME repository-mining data are organized as original per-snapshot audit files and one processed cross-snapshot summary. The processed summary is stored as:

`data/processed/knime_snapshots/knime_node_snapshot_summary.csv`.

Per-snapshot audit files are stored under `data/original/knime_snapshots/<snapshot-date>/` and include:

- `logs/checkout_<snapshot-date>.csv`
- `plugin_nodes.csv`
- `node_descriptions.csv`
- `factory_class_mappers.csv`
- `migration_rules.csv`
- `summary.csv`

Table 1: Columns in the processed KNIME node snapshot summary.

Name	Type	Short description
`snapshot_id`	string	Stable identifier for the snapshot.
`snapshot_kind`	categorical	Snapshot basis, currently date-based.
`snapshot_date`	date	Target date used for repository checkout.
`knime_version`	string	Version label or source-date context assigned by the summary script.

Name	Type	Short description
`source_basis`	categorical	Method used to obtain the source snapshot.
`repos_processed`	integer	Repositories with a checkout commit in the snapshot manifest.
`repos_skipped`	integer	Repositories without a commit at or before the target date.
`plugin_xml_files`	integer	Distinct parsed `plugin.xml` files contributing node metadata.
`repos_with_plugin_xml`	integer	Repositories contributing at least one parsed `plugin.xml`.
`registered_nodes`	integer	Ordinary node registrations in the node extension point.
`registered_nodesets`	integer	Dynamic nodeset registrations in the nodeset extension point.
`registered_total`	integer	Sum of ordinary node and nodeset registrations.
`unique_factory_classes`	integer	Distinct non-empty factory classes among ordinary node registrations.
`deprecated_nodes`	integer	Ordinary nodes with `deprecated="true"`.
`deprecated_nodesets`	integer	Nodesets with `deprecated="true"`.
`deprecated_total`	integer	Sum of deprecated ordinary nodes and deprecated nodesets.
`unique_deprecated_factory_classes`	integer	Distinct factory classes among deprecated ordinary nodes.
`deprecated_node_percent`	percent	Deprecated ordinary nodes divided by registered ordinary nodes.
`hidden_nodes`	integer	Ordinary nodes with `hidden="true"`.
`hidden_node_percent`	percent	Hidden ordinary nodes divided by registered ordinary nodes.
`deprecated_and_hidden_nodes`	integer	Ordinary nodes marked both deprecated and hidden.

Name	Type	Short description
`node_description_files`	integer	Parsed `*NodeFactory.xml` files rooted at `knimeNode`.
`description_deprecated_files`	integer	Node-description files with `deprecated="true"`.
`description_deprecated_percent`	percent	Deprecated node descriptions divided by parsed node descriptions.
`factory_class_mapper_count`	integer	Registered `NodeFactoryClassMapper` contributions.
`migration_rule_count`	integer	Registered `NodeMigrationRule` contributions.
`nodes_added_since_previous`	integer/blank	Node identity keys present in this snapshot but not the previous one.
`nodes_removed_since_previous`	integer/blank	Node identity keys present in the previous snapshot but absent here.
`nodes_newly_deprecated_since_previous`	integer/blank	Common node keys that became deprecated since the previous snapshot.
`nodes_no_longer_deprecated_since_previous`	integer/blank	Common node keys no longer marked deprecated.
`nodes_newly_hidden_since_previous`	integer/blank	Common node keys that became hidden since the previous snapshot.
`nodes_no_longer_hidden_since_previous`	integer/blank	Common node keys no longer marked hidden.
`nodes_category_changed_since_previous`	integer/blank	Common node keys whose observed category-path set changed.

5 Results

5.1 Bibliometric Evidence

The OpenAlex title-and-abstract query returned 963 article-type records. This is the first piece of evidence for the study: KNIME is visible in a sizeable scientific literature, so preservation of KNIME workflows is not only a niche platform-maintenance issue. The annual series indicates that KNIME-matching literature becomes visible in OpenAlex from the mid-2000s and grows into a sustained publication stream after 2015 (Fig. 1). The highest annual count in the current export is 104 records in 2023. Counts for 2026 are incomplete because the query was collected on June 29, 2026.

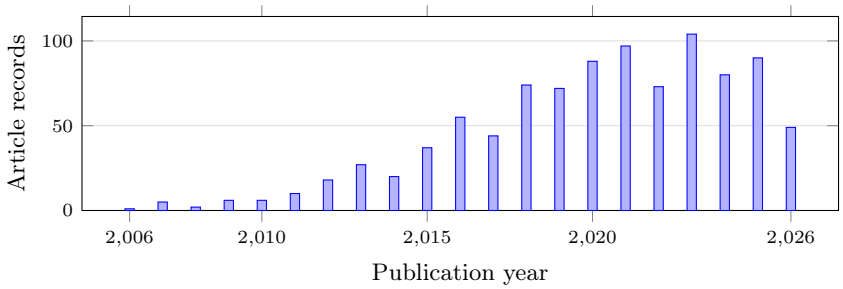

Fig. 1. OpenAlex article records matching `KNIME` in title or abstract by publication year.

The domain distribution shows that the KNIME-matching literature is not confined to one disciplinary area (Table 2). OpenAlex assigns just over half of the records to Physical Sciences, but the remaining records are spread across Social Sciences, Life Sciences, and Health Sciences. This matters for the study because workflow reproducibility risk is not only a software-engineering concern. If KNIME workflows are used across computational chemistry, biomedical analysis, education, business analytics, and related areas, then missing workflow artifacts and changing node implementations can affect published computational evidence in several research communities.

Table 2. OpenAlex domains for KNIME title-and-abstract matches.

Domain	Articles	Share
Physical Sciences	505	52.44%
Social Sciences	187	19.42%
Life Sciences	149	15.47%
Health Sciences	122	12.67%

The ten largest OpenAlex fields are led by Computer Science (350), Biochemistry, Genetics and Molecular Biology (115), Medicine (92), Engineering (71), and Social Sciences (66). These field counts show why a paper about KNIME reproducibility cannot focus only on the KNIME platform literature. The central reproducibility question is how scientific studies that use KNIME as a tool report the executable workflow, data, versions, and extensions needed for later reproduction.

Open-access metadata indicate 619 open-access records, 625 records with some full-text availability, and 421 records with a PDF signal. These signals are important because later workflow audits require access to paper text, supplements, repositories, and sometimes PDF-only workflow descriptions. The counts

are only availability indicators. They do not by themselves establish that a workflow artifact, KNIME version, input data, or extension list is available.

The heuristic role classification separates likely papers about KNIME or KNIME extensions from papers that likely use KNIME as an analysis tool. The current rules classify 156 records as about KNIME or a KNIME extension, 314 as using KNIME as a tool, 28 as about or mentioning KNIME in the title, and 465 as ambiguous OpenAlex matches. The same heuristic marks 410 records as likely using KNIME as a workflow platform. These labels are useful for triage: they identify a substantial candidate set where workflow reporting can matter, but final case-study claims require manual full-text and artifact assessment.

5.2 Top-Cited Article Assessment

The top-cited audit shows that article access itself is already a limiting condition for reproducibility assessment. Of the 100 citation-ranked KNIME-matching records, 79 have a local PDF recorded in the compact audit report. This means that the automated middle of the pipeline can evaluate most, but not all, top-cited records from local article text and extracted references. The remaining records stay in the denominator, because excluding them would overstate the assessability of the literature.

The URL and reference-page stages add a more precise view of workflow availability than article text alone. The compact report contains workflow-relevant linked resources for 22 article records, with 68 linked resources retained after filtering generic article pages, datasets, software documentation, and malformed or unrelated URLs. This result is important methodologically: the clearest evidence for whether a reader can obtain a workflow often appears on the referenced page itself, not in the sentence that cites the URL.

The LLM reference-classification step reduced the URL set to resources that could plausibly support workflow retrieval. In the compact report, most retained workflow links are classified as downloaded workflow artifacts, with only two manual-check resources and a small number of landing-page or direct-workflow references. The broader workflow inventory keeps 31 article records as download candidates, because it preserves retrieval attempts even when the compact article flag is later stricter. This separation is useful: it lets the audit distinguish "the article or reference suggests a workflow may be available" from "a workflow artifact was actually obtained locally."

The supplementary LLM flag step gives the publication-level reporting pattern around those workflow links. Seventy-five records use KNIME as a workflow, tool, interface, or implementation context. Most of these KNIME-use records describe workflow steps or nodes in text, and many show workflow screenshots or figures. However, much smaller subsets report a KNIME version, a downloadable workflow, extension dependencies, or an extension installation source. This indicates that visual or narrative workflow presentation is more common than the executable and environment context needed for later reproduction.

The workflow-download stage converts reported or linked workflow evidence into local artifact evidence. Eighteen article records yielded workflow files, archives,

or workflow directories. Four were obtained automatically and fourteen required manual download. This confirms the semi-automatic nature of the boundary step: scripted retrieval can find some artifacts, but protected pages, stale links, repository layouts, and supplement portals still require human inspection.

Table 3 summarizes the statistics-ready fields in the compact article audit report. Data and code signals are recorded separately from workflow-artifact evidence. We do not verify the availability of reported data or code in this table; we record whether the article and linked evidence report those resources.

Table 3: Flag summary for the top-cited article audit. Shares use all 100 records in the registry as the denominator.

Audit category or flag	Count	Share of 100	Interpretation
Audit records	100	100.0%	Most-cited KNIME-matching article records selected from OpenAlex
Records with local PDF	79	79.0%	Source PDF recorded in the compact audit report
Uses KNIME	75	75.0%	KNIME used as a workflow, tool, interface, or implementation context
Workflow or nodes described in text	68	68.0%	Named workflow steps, nodes, modules, or components recorded
Workflow screenshots or figures	49	49.0%	Workflow shown visually in article or supplement
Reports KNIME version	18	18.0%	Specific KNIME version value found
Reports downloadable workflows	22	22.0%	Executable workflow files provided or linked
Reports extension/plugin dependencies	23	23.0%	KNIME extension or node-library dependency reported
Reports extension installation source	4	4.0%	Update site, project URL, or installation source reported
Provides code or scripts	20	20.0%	Code repository, scripts, or software code reported
Reports input-data availability	66	66.0%	Data availability stated, with or without direct URL
Direct input-data resource	20	20.0%	Direct data URL or resource pointer recorded

Table 4: Workflow-reporting and retrieval signals among the 75 assessed KNIME-use cases.

Audit flag	Count	Share of KNIME-use cases
Uses KNIME	75	100.0%
Workflow or nodes described in text	68	90.7%
Workflow screenshots or figures	49	65.3%
Reports KNIME version	18	24.0%
Reports extension/plugin dependencies	23	30.7%
Reports extension installation source	4	5.3%
Reports downloadable workflows	22	29.3%
Articles with successfully downloaded workflows	18	24.0%

For workflow retrieval, the authoritative data source is the workflow inventory derived from the compact audit report and the local workflow directories.

Table 5 summarizes the recorded artifact-retrieval outcomes used for manuscript inspection. In this table, the *Success* column marks whether a KNIME workflow artifact was actually obtained: **X** means that a workflow file, archive, or workflow directory was retrieved, while **-** means that no KNIME workflow artifact was obtained or confirmed. Failed or incomplete retrievals are recorded separately from article-level claims that a workflow was reported or linked.

Table 6 then reports the manual current-KNIME opening and execution checks for the downloaded article records. Here, the *Executed* column has a stricter meaning: **X** means that at least one workflow for the article opened and executed successfully in the tested environment, while **-** means that the workflow opened but did not have a confirmed successful end-to-end run, or failed during execution. Five article records had at least one workflow execute successfully in the recorded manual checks: PAINS, Webinar Pricing Analytics, ImageJ ecosystem integration, high-content organelle trafficking, and GediNET [21, 23, 2, 17, 19]. Other opened workflows did not have a confirmed successful end-to-end run in this manual check. This does not show that those workflows are unreproducible: they may require additional configuration, dependencies, or repair before a successful run. Across the opened workflows, the screenshots also show deprecated or legacy nodes, but we treat this as qualitative compatibility evidence here rather than counting visible deprecated nodes. The screenshots are archived under each workflow directory's `opened/` subdirectory in `data/original/workflows/`.

Table 5: Workflow retrieval results for article records with attempted or recorded artifact retrieval outcomes.

#	Article record	Success	Retrieval note
1	PAINS workflows, DOI `10.1002/minf.201100076`	**X**	Automated myExperiment page and download requests returned HTTP 403 HTML, but we manually obtained both RDKit and Indigo workflow ZIP archives. Both contain `workflow.knime` files.
2	Chemical data curation workflow, DOI `10.1186/s13321-018-0315-6`	**X**	GitHub master archive was downloaded after retry and passed ZIP integrity checking. KNIME workflow files were found in the archive.
3	ASD genes prediction workflow, DOI `10.1371/journal.pone.0208626`	**X**	GitHub archive passed ZIP integrity checking. An inner KNIME workflow archive was extracted to a workflow directory.
4	Bitterness/sweetness classifier workflow, DOI `10.3389/fchem.2018.00093`	-	The original project page redirected to VirtualTaste, and the redirected page returned HTTP 404. No workflow archive was retrieved.
5	Medicinal chemistry filters workflow, DOI `10.3390/encyclopedia3020035`	**X**	GitLab archive passed ZIP integrity checking and contained KNIME workflow files. One KNIME Hub URL returned a page, while an older generic Hub URL returned 404 JSON.
6	MS-ready structures repository, DOI `10.1186/s13321-018-0299-2`	-	The GitHub archive passed ZIP integrity checking, but the extracted repository contained only `LICENSE` and `README.md`. No `.knwf` or `workflow.knime` file was found.
7	KNIME-OMICS reference, DOI `10.1016/j.jprot.2016.08.002`	-	The referenced GitHub organization page returned HTTP 404. No workflow archive was retrieved.

#	Article record	Success	Retrieval note
8	QSAR tools workflow, DOI 10.1080/07391102.2018.1456975	-	The referenced teqip.jdvu.ac.in/QSAR_Tools/ URL failed DNS resolution. No workflow archive was retrieved.
9	Webinar Pricing Analytics workflow, DOI 10.1016/j.jbusres.2021.08.036	X	We manually downloaded the KNIME workflow file and added local KNIME opening screenshots.
10	Verhaar scheme workflow, DOI 10.1016/j.chemosphere.2015.06.009	-	The article reports a supplementary KNIME workflow. Standard Elsevier supplement URLs resolved as Excel files, but no direct KNIME workflow archive URL was confirmed.
11	Spontaneous tail coiling workflow, DOI 10.1016/j.ntt.2020.106918	X	We manually downloaded the KNIME workflow file and added local KNIME opening screenshots.
12	Caco-2 permeability prediction workflow, DOI 10.3390/pharmaceutics14101998	X	We manually downloaded the KNIME workflow file and added local KNIME opening screenshots.
13	KniMet workflow, DOI 10.1007/s11306-018-1349-5	X	The Zenodo DOI resolved to record 1196407. KniMet-master.zip was downloaded and contains KniMet.knwf.
14	NGS sample workflow, DOI 10.1093/bioinformatics/btr478	X	We manually downloaded the myExperiment archive after automated requests were blocked by HTTP 403. The ZIP contains a KNIME workflow directory with saved data.
15	ImageJ ecosystem workflows, DOI 10.3389/fcomp.2020.00008	X	Six KNIME Hub artifact metadata endpoints returned signed URLs and the corresponding .knwf files were downloaded. One recorded reference appears stale or unavailable.
16	Multi-template matching repository, DOI 10.1186/s12859-020-3363-7	-	The GitHub archive was downloaded, but it contained documentation/images only and no .knwf, workflow.knime, or KNIME workflow archive. A guessed NodePit URL returned HTTP 404.
17	GediNET workflows, DOI 10.1038/s41598-022-24421-0	X	GitHub master archive was downloaded and contains two .knwf files. The recorded KNIME Hub short link could not be retrieved.
18	Nuclear receptor ligand workflow, DOI 10.1002/minf.201400188	-	The article reports supplementary workflow material, but Wiley article and guessed supplementary download endpoints returned Cloudflare challenge pages. Crossref metadata did not expose a supplementary workflow URL.
19	High-content organelle trafficking workflow, DOI 10.1038/sdata.2018.241	X	The figshare collection DOI resolved through the figshare API. The default KNIME workflow file was downloaded as KNIME workflow.zip.
20	Knime4Bio NOTCH2 workflow, DOI 10.1093/bioinformatics/btr554	X	The article reports that the workflow was posted on myExperiment. The local manual artifact is The_NOTCH2_workflow-v1.zip, which contains a KNIME workflow directory.

Table 6: Manual opening and execution results for downloaded KNIME workflow artifacts.

#	Article record	Executed	Manual KNIME result
1	PAINS workflows, DOI 10.1002/minf.201100076	**X**	RDKit and Indigo workflow ZIPs were tested. At article-record level, at least one workflow opened and executed successfully. The separate Indigo workflow opened but was blocked by a missing Indigo KNIME Integration node.
2	Chemical data curation workflow, DOI 10.1186/s13321-018-0315-6	-	The CompTox rebuilt workflow file opened, but execution failed inside a metanode during the manual test.
3	ASD genes prediction workflow, DOI 10.1371/journal.pone.0208626	-	The extracted ASD genes prediction workflow opened, but execution failed in an R node because required R packages and functions were unavailable in the test environment.
4	Medicinal chemistry filters workflow, DOI 10.3390/encyclopedia3020035	-	The medicinal chemistry filters workflow file opened, but a successful end-to-end execution was not established in the available notes.
5	Webinar Pricing Analytics workflow, DOI 10.1016/j.jbusres.2021.08.036	**X**	The Webinar Pricing Analytics workflow file opened and executed successfully in the current local KNIME environment.
6	Spontaneous tail coiling workflow, DOI 10.1016/j.ntt.2020.106918	-	The spontaneous tail coiling measurement workflow file opened in the current local KNIME environment, but a successful end-to-end execution was not established in the available notes.
7	Caco-2 permeability prediction workflow, DOI 10.3390/pharmaceutics14101998	-	The manual Caco-2 prediction `.knwf` file opened in the current local KNIME environment, but a successful end-to-end execution was not established in the available notes.
8	KniMet workflow, DOI 10.1007/s11306-018-1349-5	-	The KniMet workflow from the Zenodo archive opened in the current local KNIME environment, but successful execution was not established in the available notes.
9	NGS sample workflow, DOI 10.1093/bioinformatics/btr478	-	The NGS sample workflow archive with saved data opened in the current local KNIME environment, but successful execution was not established in the available notes.
10	ImageJ ecosystem workflows, DOI 10.3389/fcomp.2020.00008	**X**	Six downloaded KNIME Hub `.knwf` files opened and executed normally in our test environment after installing additional required KNIME/ImageJ-related plugins.
11	GediNET workflows, DOI 10.1038/s41598-022-24421-0	**X**	The GediNET GitHub archive opened in the current local KNIME environment and the workflow was run successfully in the manual check.
12	High-content organelle trafficking workflow, DOI 10.1038/sdata.2018.241	**X**	`KNIME workflow.zip` opened and executed normally in our test environment after installing additional required plugins.
13	Knime4Bio NOTCH2 workflow, DOI 10.1093/bioinformatics/btr554	-	`The_NOTCH2_workflow-v1.zip` opened in the current local KNIME environment, but a successful end-to-end execution was not established in the available notes.

5.3 Repository-Level Analysis

The repository-mining results explain why weak workflow reporting becomes more costly over time. Deprecated nodes are not an isolated metadata corner

case in the KNIME source tree. At the earliest available source-code baseline, 2018-04-03, 193 of 1301 registered ordinary nodes were marked as deprecated, or 14.83% (Table 7). By the KNIME Analytics Platform 5.0.0 source-date anchor on 2023-02-22, the count had increased to 433 of 1442 ordinary nodes, or 30.03%. The current local source-date snapshot on June 28, 2026, contains 502 deprecated ordinary nodes out of 1506, or 33.33%. These figures show that the deprecated-node share approximately doubled between the 2018 baseline and the 2026 local snapshot.

The annual checkpoint table gives the same trend at regular intervals (Table 8). The number of repositories with checkout commits rises from 44 in 2018 to 90 in 2026, so early and late totals should not be compared as if the mined source corpus were fixed. Nevertheless, the deprecated-node share rises from 14.83% in 2018 to 33.47% at the 2026 annual checkpoint. Hidden ordinary nodes remain much less frequent than deprecated ordinary nodes, but they increase from 2 in 2018 to 24 in 2026. Deprecated dynamic nodesets appear from 2020 onward and remain at 2 in the annual checkpoints through 2026.

Table 7. Selected KNIME repository-mining snapshots.

Date	Version context	Nodes	Deprecated nodes	Deprecated share
2018-04-03	KNIME Analytics Platform 3.5.3 source-date anchor	1301	193	14.83%
2019-12-05	KNIME Analytics Platform 4.1.0 source-date anchor	1191	227	19.06%
2023-02-22	KNIME Analytics Platform 5.0.0 source-date anchor	1442	433	30.03%
2026-03-03	KNIME Analytics Platform 5.11.0 source-date anchor	1503	503	33.47%
2026-06-28	Current local source-date snapshot	1506	502	33.33%

Table 8. Annual KNIME repository-mining checkpoint statistics.

Year	Snapshot date	Repositories	Nodes	Deprecated nodes	Deprecated nodesets	Hidden nodes	Deprecated share
2018	2018-04-03	44	1301	193	0	2	14.83%
2019	2019-01-01	45	1500	248	0	2	16.53%
2020	2020-01-01	54	1200	228	2	2	19.00%
2021	2021-01-01	60	1366	398	2	5	29.14%
2022	2022-01-01	67	1424	386	2	8	27.11%
2023	2023-01-01	72	1428	433	2	10	30.32%
2024	2024-01-01	80	1434	436	2	9	30.40%
2025	2025-01-01	86	1488	446	2	23	29.97%
2026	2026-01-01	90	1509	505	2	24	33.47%

The transition table indicates that node deprecation changes occur in bursts rather than at a constant rate (Table 9). The largest annual increases in newly

Table 9. Annual KNIME repository-mining transition statistics.

Year	Added	Removed	Newly deprecated	No longer deprecated	Category changed
2018	–	–	–	–	–
2019	194	22	16	0	6
2020	11	0	0	0	4
2021	140	1	80	0	111
2022	69	6	17	29	17
2023	56	4	61	0	67
2024	17	7	4	0	3
2025	36	0	9	0	1
2026	52	2	60	0	3

deprecated node identities occur at the 2021, 2023, and 2026 checkpoints, with 80, 61, and 60 newly deprecated node keys, respectively. The 2022 checkpoint is the only annual checkpoint in which a substantial number of common node keys are no longer marked deprecated: 29. Category-path changes are also concentrated in a few years, especially 2021 and 2023. Because these transition counts use metadata-level node identity keys, they should be read as evidence of source metadata evolution, not as direct evidence that published workflows fail to open or execute.

These repository-level results directly address the second research question: how KNIME node deprecation evolved over time. They also provide necessary context for the workflow-level questions, because a published workflow that depends on older node identities may encounter deprecated, hidden, removed, or migrated components in a later KNIME version. However, the tables do not answer how often uploaded or executed workflows use deprecated nodes. That question requires workflow-level corpora and execution traces beyond repository metadata.

Taken together, the evidence gives the intended picture of the study. First, KNIME is visible in a broad and sustained scientific literature. Second, influential papers that use or mention KNIME rarely provide the full workflow-preservation package needed for later reproduction: executable workflow files, KNIME versions, extension context, data, and code. Third, the workflow experiment shows the practical consequence of this gap: 18 article records yielded workflow artifacts or workflow directories, workflows from all 18 opened in the current local KNIME environment, and five article records had at least one workflow execute successfully. Fourth, KNIME's source metadata continues to evolve, and roughly one third of ordinary registered nodes are marked deprecated in recent source snapshots. The result is not a claim that all KNIME workflows fail. The result is a reproducibility risk: as KNIME evolves, papers that preserve only workflow images or textual descriptions become harder to diagnose, migrate, and reproduce.

6 Discussion

To our knowledge, this is the first focused empirical study connecting KNIME workflow reporting in scientific publications with KNIME source-code evolution. The contribution is intentionally narrower than a full reproducibility benchmark. We do not estimate the failure rate of all published KNIME workflows. Instead, we show a reproducibility risk pattern: KNIME is used in a visible scientific literature, influential papers often do not publish executable workflow artifacts, successful current-version execution was confirmed for five article records in the recorded manual checks, and the KNIME node ecosystem has accumulated a substantial deprecated-node surface.

The results suggest three practical implications. First, a published workflow image is not an executable artifact. Several highly cited papers provide figures or textual descriptions of connected KNIME nodes, but this is not enough to recover node settings, extension versions, input paths, or migration requirements. Second, reporting a KNIME version is useful but incomplete. Workflows may also depend on third-party extension update sites and node libraries, as shown by the PAINS case where the RDKit workflow could be updated and executed while the Indigo workflow was blocked by a missing extension. Third, repository-level deprecation is a useful warning signal. It does not prove that a workflow fails, but it identifies a compatibility surface that grows as the platform evolves. Without careful workflow preservation and version reporting, this growth makes old workflows harder to inspect and repair.

The next development of this study should move from paper-level evidence to workflow-level evidence. One direction is to create a larger corpus of public KNIME workflows from supplements, myExperiment, KNIME Hub, GitHub, and other repositories, then test whether they open in current KNIME versions and which deprecated, hidden, migrated, or missing nodes they contain. A second direction is to link deprecated nodes to migration metadata and replacement nodes in the KNIME source tree, so that workflow warnings can distinguish low-risk deprecations from nodes that require unavailable extensions or manual repair. A third direction is to preserve reproducibility environments for selected case studies by recording operating system, KNIME version, installed extensions, update sites, input data, import warnings, execution errors, and output differences.

Tools and services outside the present evidence base can provide the next layer of usage evidence. `knime2py` can help identify KNIME workflows that users try to translate or operationalize outside KNIME [7], and `k2pweb.org` can provide anonymized aggregate evidence about real uploaded or executed workflows [6]. These sources would address a question that the current paper deliberately leaves open: how often researchers and analysts encounter deprecated KNIME nodes in practice, not only how often deprecated nodes appear in the source repository or in selected published artifacts.

References

1. Berthold, M.R., Cebron, N., Dill, F., Gabriel, T.R., Kötter, T., Meinl, T., Ohl, P., Sieb, C., Thiel, K., Wiswedel, B.: KNIME: The Konstanz information miner. In: Data Analysis, Machine Learning and Applications, pp. 319–326. Springer (2008). https://doi.org/10.1007/978-3-540-78246-9_38
2. Dietz, C., Rueden, C.T., Helfrich, S., Dobson, E.T.A., Horn, M., Eglinger, J., Evans, E.L., McLean, D.T., Novitskaya, T., Ricke, W.A., Sherer, N.M., Zijlstra, A., Berthold, M.R., Eliceiri, K.W.: Integration of the ImageJ ecosystem in KNIME analytics platform. Frontiers in Computer Science **2**, 8 (2020). https://doi.org/10.3389/fcomp.2020.00008
3. Fu, X., Liu, L., Guan, W.W., Kalra, Y., Bao, S., Kötter, T., Sturm, K.: Advancing replicable and reproducible GIScience: an approach with KNIME. Cartography and Geographic Information Science **53**(3), 270–290 (2025). https://doi.org/10.1080/15230406.2024.2446556
4. GROBID contributors: GROBID: Generation of bibliographic data. `https://github.com/kermitt2/grobid` (2026), version 0.9.0-crf Docker image; accessed July 12, 2026
5. Hall, M., Frank, E., Holmes, G., Pfahringer, B., Reutemann, P., Witten, I.H.: The WEKA data mining software: An update. ACM SIGKDD Explorations Newsletter **11**(1), 10–18 (2009). https://doi.org/10.1145/1656274.1656278
6. Kaplan, V.: k2p-web: KNIME to Python/Jupyter converter. `https://k2pweb.org/` (2026), accessed July 1, 2026
7. Kaplan, V.: knime2py: KNIME to Python workbook. `https://github.com/vitalii-kaplan/knime2py` (2026), accessed July 1, 2026
8. Kaplan, V.: reproducibility-risks-article: Replication repository for KNIME workflow reproducibility-risk study. `https://github.com/vitalii-kaplan/reproducibility-risks-article` (2026), accessed July 1, 2026
9. KNIME AG: Installing extensions and integrations. `https://docs.knime.com/ap/latest/analytics_platform_user_guide/index.html\#installing-extensions-and-integrations` (2026), accessed June 29, 2026
10. KNIME AG: KNIME Analytics Platform User Guide. `https://docs.knime.com/ap/latest/analytics_platform_user_guide/index.html` (2026), accessed June 29, 2026
11. KNIME AG: KNIME Core NodeFactoryClassMapper Extension Point Schema. `https://github.com/knime-oss/knime-core/blob/3885b31a49d047401a076981f9bd4c964b4a97a7/org.knime.core/schema/NodeFactoryClassMapper.exsd` (2026), accessed June 29, 2026
12. KNIME AG: KNIME Workbench Repository Node Extension Point Schema. `https://github.com/knime-oss/knime-workbench/blob/c94203746f98ab38cccea54e3497d62c9adee9e1/org.knime.workbench.repository/schema/Node.exsd` (2026), accessed June 29, 2026
13. KNIME AG: KNIME Workflow Migration NodeMigrationRule Extension Point Schema. `https://github.com/knime-oss/knime-database/blob/c4a3356985edf4383ad0a3107658851e6c49fdad/org.knime.workflow.migration/schema/NodeMigrationRule.exsd` (2026), accessed June 29, 2026
14. Kurtzer, G.M., Sochat, V., Bauer, M.W.: Singularity: Scientific containers for mobility of compute. PLOS ONE **12**(5), e0177459 (2017). https://doi.org/10.1371/journal.pone.0177459

15. Microsoft: Playwright for Python. https://playwright.dev/python/ (2026), version 1.61.0; accessed July 12, 2026
16. OpenAI: OpenAI API Documentation: Chat completions. https://platform.openai.com/docs/api-reference/chat (2026), accessed July 12, 2026
17. Pal, A., Glaß, H., Naumann, M., Kreiter, N., Japtok, J., Sczech, R., Hermann, A.: High content organelle trafficking enables disease state profiling as powerful tool for disease modelling. Scientific Data **5**(1), 180241 (2018). https://doi.org/10.1038/sdata.2018.241
18. Priem, J., Piwowar, H., Orr, R.: OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. arXiv preprint arXiv:2205.01833 (2022). https://doi.org/10.48550/arXiv.2205.01833, https://arxiv.org/abs/2205.01833
19. Qumsiyeh, E., Showe, L.C., Yousef, M.: GediNET for discovering gene associations across diseases using knowledge based machine learning approach. Scientific Reports **12**(1), 19955 (2022). https://doi.org/10.1038/s41598-022-24421-0
20. Samuel, S., Mietchen, D.: Computational reproducibility of Jupyter notebooks from biomedical publications. GigaScience **13** (2024). https://doi.org/10.1093/gigascience/giad113
21. Saubern, S., Guha, R., Baell, J.B.: KNIME workflow to assess PAINS filters in SMARTS format: Comparison of RDKit and Indigo cheminformatics libraries. Molecular Informatics **30**(10), 847–850 (2011). https://doi.org/10.1002/minf.201100076
22. Tahir, A., Rasheed, S., Dietrich, J., Hashemi, N., Zhang, L.: Test flakiness' causes, detection, impact and responses: A multivocal review. Journal of Systems and Software **206**, 111837 (2023). https://doi.org/10.1016/j.jss.2023.111837
23. Villarroel Ordenes, F., Silipo, R.: Machine learning for marketing on the KNIME hub: The development of a live repository for marketing applications. Journal of Business Research **137**, 393–410 (2021). https://doi.org/10.1016/j.jbusres.2021.08.036
24. Walker, R., Slingsby, A., Dykes, J., Xu, K., Wood, J., Nguyen, P.H., Stephens, D., Wong, B.L.W., Zheng, Y.: An extensible framework for provenance in human terrain visual analytics. IEEE Transactions on Visualization and Computer Graphics **19**(12), 2139–2148 (2013). https://doi.org/10.1109/TVCG.2013.132